

COMPSCI 589 Homework 3 - Spring 2026

Due **March 30**, 11:55 pm Eastern Time

Instructions

- This homework assignment consists of a programming part.
- While you may discuss problems with your peers (e.g., to discuss high-level approaches), you must answer the questions on your own and implement all solutions independently. In your submission, do explicitly list all students with whom you discussed this assignment.
- Submissions must be typed. You must use $\text{\LaTeX}$ to prepare your submission. Handwritten and scanned submissions will not be accepted.
- The written report, including your answers, plots, and analysis, should be submitted on Gradescope as a PDF via the appropriate submission link. The source code should be submitted as a single .zip file through the separate programming assignment link on Gradescope.
- We strongly encourage using Python 3 for your homework code. You may use other languages. In either case, you *must* provide clear instructions for running your code and reproducing your experiments.
- You *may not* use any machine learning-specific libraries in your code, e.g., TensorFlow, PyTorch, or machine learning algorithms implemented in scikit-learn (though you *may* use other functions provided by this library, such as one that splits a dataset into training and testing sets). You may also use libraries such as numpy and matplotlib. If you are not certain whether a specific library is allowed, do ask us.
- All submissions will be checked for plagiarism using two independent plagiarism-detection tools. Renaming variable or function names, moving code within a file, etc., are all strategies that *do not* fool the plagiarism-detection tools we use. Your code will also be compared with submissions from your classmates and with publicly available implementations online.
- If you get caught, all penalties mentioned in the syllabus *will* be applied—which may include directly failing the course with a letter grade of “F”.

→ Before starting this homework, please review this course’s policies on plagiarism by reading the corresponding section of the syllabus.

- The automated system will not accept assignments after 11:55 pm on March 30.
- The TeX file for this homework (which you should use to write your solutions in $\text{\LaTeX}$), as well as the datasets you will investigate, are available on Canvas under “Homework #3 - Supporting Files”. See below for more details.

Programming Assignment [100 Points, total]

In this homework, you will be implementing the Random Forest algorithm. **Notice that you may not use existing machine learning code for this problem: you must implement the learning algorithms entirely on your own and from scratch.**

In this assignment, you will:

- Leverage your implementation of the standard Decision Tree algorithm, based on the Information Gain splitting criterion, to implement a Random Forest classifier.

Important: You must reuse *your* previous implementation of the Decision Tree algorithm.

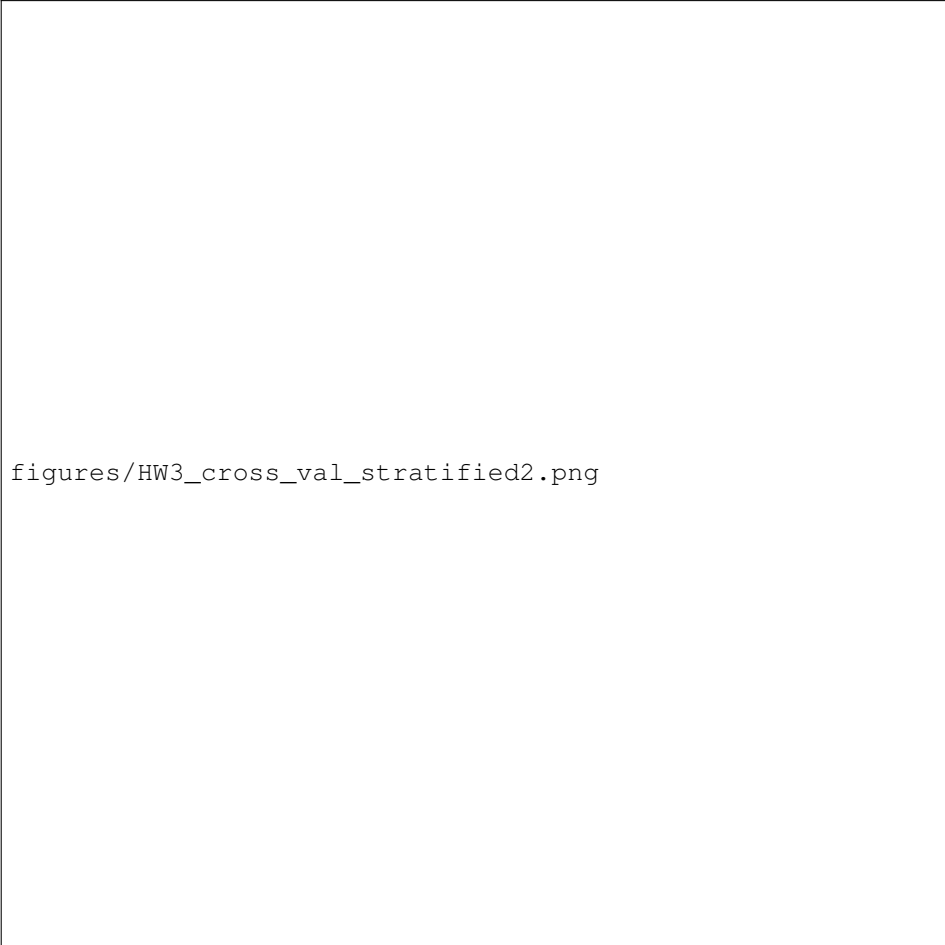
Important: Your implementation of the Decision Tree algorithm should support both categorical *and* numerical attributes. When determining the numerical threshold to use when splitting numerical nodes, you may (i) use the average of that attribute's values in the partition of instances associated with the current node; or (ii) use the more sophisticated (and “powerful”) technique discussed in Lecture 05.

- Implement a procedure to create bootstrap datasets by sampling (*with replacement*) from the current training set to construct the datasets based on which each tree in the random forest will be trained.
- Implement a mechanism to select (whenever a node is being split) m attributes at random from the complete set of attributes of an instance. You should then split the node based on the attribute with the best Information Gain. As a heuristic, we usually use $m \approx \sqrt{X}$, where X is the total number of attributes in each instance of the dataset.
- Implement a majority voting mechanism that takes into account the predictions made by each tree in the ensemble to classify new instances.
- Implement the *stratified cross-validation* technique to evaluate the generalization capability of the random forest. For an example of how this technique works, see Figure 1. We recommend using $k = 5$ folds. Of these, $k - 1$ folds will be used to train a random forest composed of $ntree$ trees. Here, $ntree$ **will be the main adjustable hyperparameter of your algorithm.**

Note: In cross-validation, the remaining fold (the one not used during training) will be used to evaluate the performance of the random forest itself (i.e., not the performance of any individual tree separately!).

Another way to describe how cross-validation works in our setting is as follows: at each iteration of the cross-validation process, you will train not only *one* decision tree, but multiple ($ntree$) decision trees, all of which will be combined to form your random forest. You will then repeat this process, training and evaluating k random forests (one for each of the k cross-validation iterations). The final cross-validation performance you should return is the average of the performances computed for each of the folds.

- Evaluate the impact the number of trees (i.e., the $ntree$ hyperparameter) has on the performance of your random forest.



figures/HW3_cross_val_stratified2.png

Figure 1: Example of how to construct *stratified* folds to perform k -fold cross-validation. Here, we assume two classes: **M** and **F**. Notice how the complete dataset (shown in the leftmost figure) is not balanced: approximately 3/4 of its instances belong to the class **M**, and approximately 1/4 of its instances belong to the class **F**. We wish to approximately preserve these proportions (i.e., the proportion of instances of class **M** and the proportion of instances of class **F**) in each of the $k = 5$ folds. To do this, you should first split the original dataset, D , into two: one containing only instances of class **M** (sub-dataset D_M), and another one containing only instances of class **F** (sub-dataset D_F). Then, given the total number of instances in the original dataset (D) and the number of folds you are using (k), you should be able to determine the size of each fold. Let us assume that each fold, in this example, should have 50 instances. To construct each fold, you should draw random instances from D_M and D_F . In this example, 3/4 of the instances that you add to the current fold should come from D_M , and 1/4 of the instances that you add to the current fold should come from D_F . Notice that because the folds should be disjoint (i.e., an instance should appear in only one fold), after drawing an instance either from D_M or D_F , it should be removed from such a set to ensure that it will not be accidentally included as part of another fold. (Source: [Stack Exchange](#)).

1 Deliverables and Datasets

The main objective of this homework is to study the impact of the number of trees in a random forest (i.e., the *ntree* hyperparameter) on its performance. **When constructing each tree in the ensemble, you are free to choose among several stopping criteria.** You may, for instance:

- Stop splitting nodes whenever the data partition associated with the current node contains fewer

than n instances, for some value of n that you choose empirically. We will refer to this as the `minimum_size_for_split` criterion.

- Stop splitting nodes whenever the information gain resulting from a split is not sufficiently high (even if it is slightly greater than zero). You are free to choose empirically the minimum information gain you would like to use. We will refer to this as the `minimum_gain` criterion.
- You may stop splitting whenever the current branch of the tree becomes too deep, that is, when its depth exceeds some pre-defined value that you select empirically. We will refer to this as the `maximum_depth` stopping criterion.

Important: In all questions below, you should explicitly mention which criterion or criteria you used and the corresponding hyperparameter(s). You should try different options to identify the setting that produces the best possible performance for your algorithm.

In this homework, you will be analyzing two datasets:

(1) **The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset.** The goal here is to train a classifier capable of predicting whether a breast mass is malignant or benign, based on features computed from a digitized image of a breast mass. The dataset is composed of 569 instances. Each instance is described by 30 *numerical* attributes, and there are two classes.

(2) **The Loan Eligibility Prediction Dataset.** Here, the goal is to automatically (and accurately) predict whether a given person should qualify for a loan. Each instance is described by 11 attributes, including the gender of the applicant, whether the applicant is married, their income, information about their credit history, etc. The binary target class to be predicted is whether a given applicant's request for a loan was approved or not. Notice that this dataset contains 7 *categorical* attributes and 4 *numerical* attributes. There is a total of 480 instances in the dataset.

Note: The datasets above can be found in the “*Supporting Files*” zip file for this homework on Canvas.

2 Questions

For each dataset, you should:

1. Train a random forest and evaluate it using the stratified cross-validation technique discussed above. You should train random forests with different values of the *ntree* parameter: 1, 5, 10, 20, 30, 40, and 50.

A random forest has been trained and evaluated using stratified cross-validation technique. The random forest has been trained with different values of *ntree* parameters. The graphs for various metrics like Accuracy, Precision, Recall and F1 Score and their explanation are discussed in the below questions.

The below stopping criteria were used while validating the random forest model: maximum depth = 10, minimum size for split = 5 and information gain ≤ 0.01

2. For each value of *ntree*, you should measure the accuracy, precision, recall, and F1 score of the resulting random forest.

The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

Accuracy, Precision, Recall and F1 Score of the Random Forest for the Wisconsin Breast Cancer Diagnostic (WDBC) Dataset for each value of *ntree*

```
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 1
{'Accuracy': 0.90858, 'Precision': 0.92389, 'Recall': 0.932707, 'F1 Score': 0.9278}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 5
{'Accuracy': 0.947272, 'Precision': 0.948356, 'Recall': 0.969131, 'F1 Score': 0.958454}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 10
{'Accuracy': 0.959569, 'Precision': 0.964104, 'Recall': 0.971948, 'F1 Score': 0.967903}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 20
{'Accuracy': 0.963109, 'Precision': 0.962207, 'Recall': 0.980399, 'F1 Score': 0.970987}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 30
{'Accuracy': 0.956245, 'Precision': 0.958709, 'Recall': 0.972144, 'F1 Score': 0.965142}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 40
{'Accuracy': 0.964879, 'Precision': 0.967256, 'Recall': 0.977582, 'F1 Score': 0.972293}
The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - ntree parameter = 50
{'Accuracy': 0.966618, 'Precision': 0.959616, 'Recall': 0.988732, 'F1 Score': 0.973843}
```

Figure 2: Performance metrics - Accuracy, Precision, Recall and F1 Score for WDBC dataset for different values of *ntree*

The Loan Eligibility Prediction Dataset

Accuracy, Precision, Recall and F1 Score of the Random Forest for the Loan Eligibility Prediction Dataset for each value of *ntree*

```
The Loan Eligibility Prediction Dataset - ntree=1
{'Accuracy': 0.720463, 'Precision': 0.742887, 'Recall': 0.915468, 'F1 Score': 0.818861}
The Loan Eligibility Prediction Dataset - ntree=5
{'Accuracy': 0.720744, 'Precision': 0.717938, 'Recall': 0.984894, 'F1 Score': 0.830116}
The Loan Eligibility Prediction Dataset - ntree=10
{'Accuracy': 0.758291, 'Precision': 0.749288, 'Recall': 0.979014, 'F1 Score': 0.848578}
The Loan Eligibility Prediction Dataset - ntree=20
{'Accuracy': 0.741536, 'Precision': 0.730413, 'Recall': 0.99697, 'F1 Score': 0.842689}
The Loan Eligibility Prediction Dataset - ntree=30
{'Accuracy': 0.789391, 'Precision': 0.776486, 'Recall': 0.978969, 'F1 Score': 0.865744}
The Loan Eligibility Prediction Dataset - ntree=40
{'Accuracy': 0.737584, 'Precision': 0.729269, 'Recall': 0.993939, 'F1 Score': 0.840567}
The Loan Eligibility Prediction Dataset - ntree=50
{'Accuracy': 0.748134, 'Precision': 0.740327, 'Recall': 0.981909, 'F1 Score': 0.843873}
```

Figure 3: Performance metrics - Accuracy, Precision, Recall and F1 Score for Loan dataset for different values of *ntree*

3. You should then create, for each dataset and for each of the four metrics described above, a graph that shows the value of *ntree* on the x-axis and the value of the corresponding metric on the y-axis. For example, you should include one graph showing the accuracy of the random forest as a function of *ntree* for Dataset #1, another graph showing the precision of the random forest as a function of *ntree* for Dataset #1, and similar graphs for the other metrics and for Dataset #2.

Important: The analyses mentioned above should result in a total of **8 graphs**. Please indicate clearly in the caption of each figure/graph which results are shown in that graph.

The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

Accuracy of the random forest as a function of $ntree$ for the Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

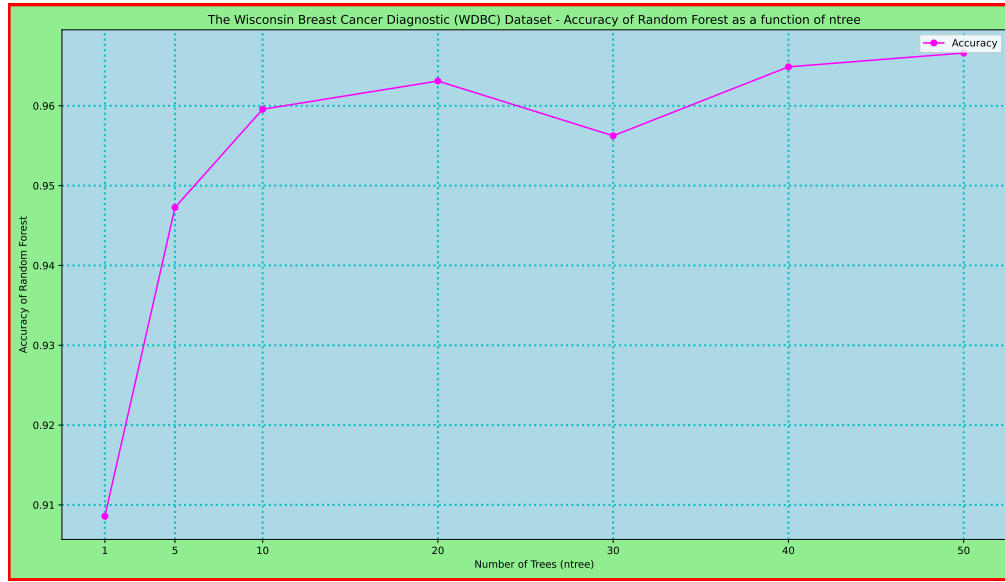


Figure 4: The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - Accuracy of the random forest as a function of $ntree$

Precision of the random forest as a function of $ntree$ for the Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

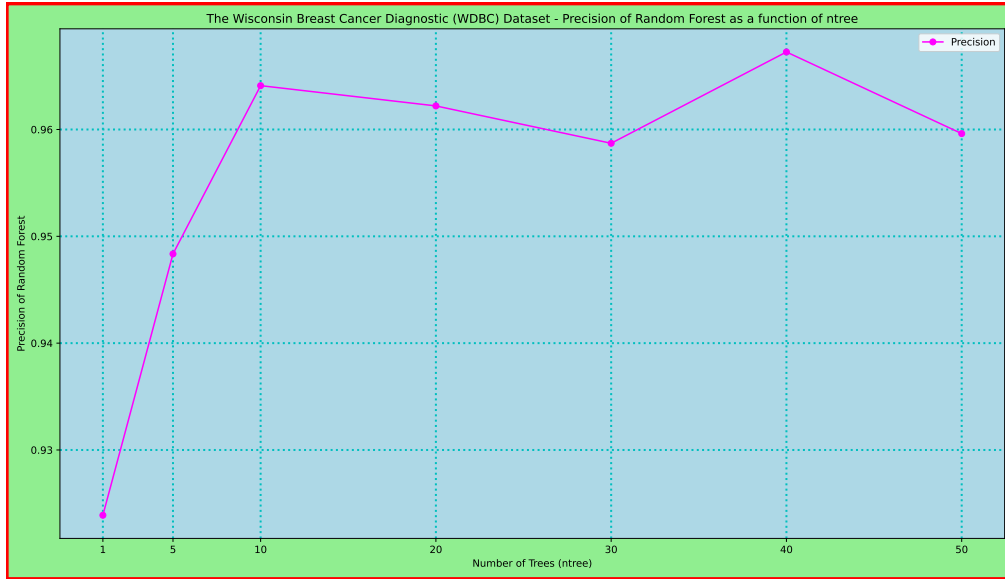


Figure 5: The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - Precision of the random forest as a function of $ntree$

Recall of the random forest as a function of $ntree$ for the Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

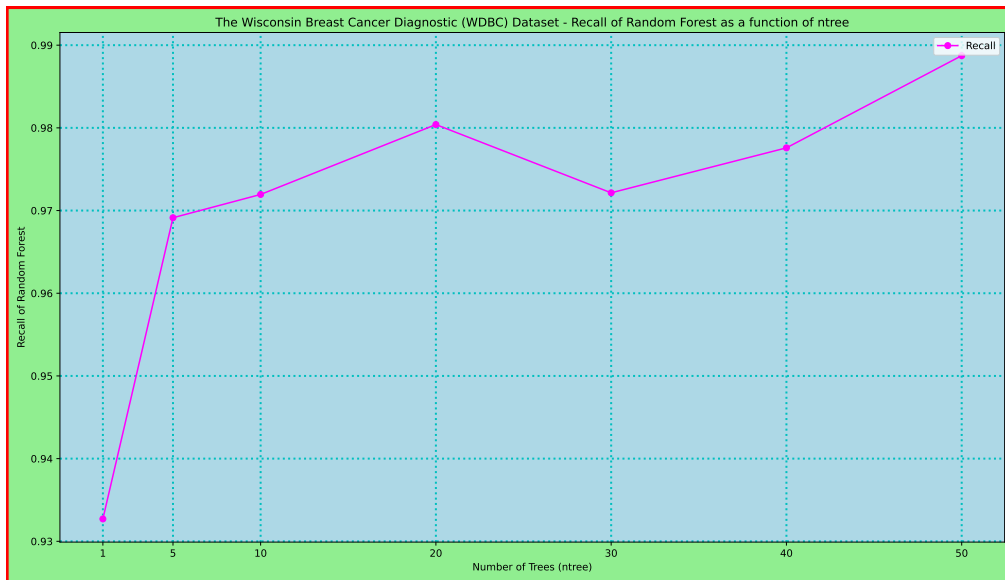


Figure 6: The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - Recall of the random forest as a function of $ntree$

F1 Score of the random forest as a function of $ntree$ for the Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

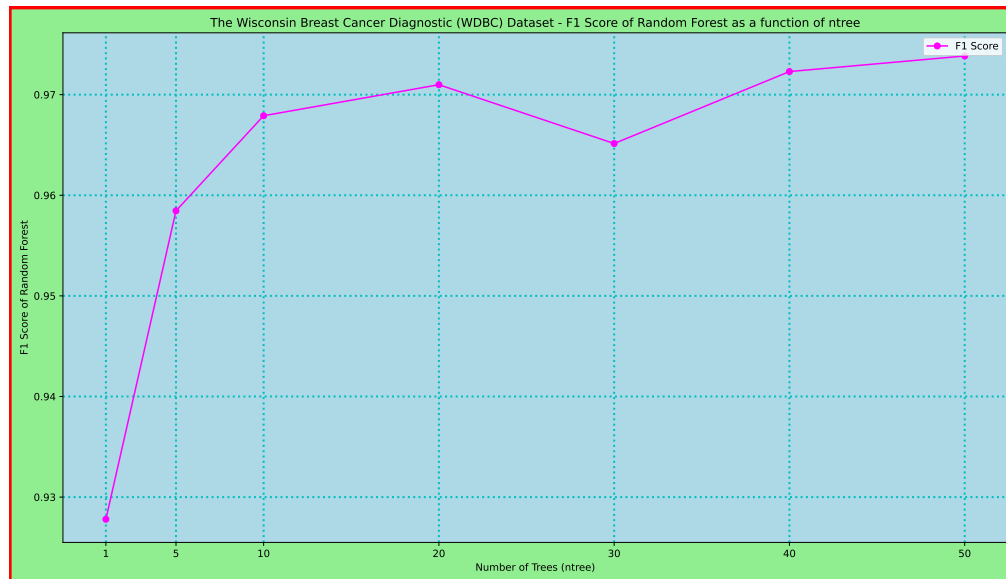


Figure 7: The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset - F1 Score of the random forest as a function of $ntree$

The Loan Eligibility Prediction Dataset

Accuracy of the random forest as a function of $ntree$ for the Loan Eligibility Prediction Dataset

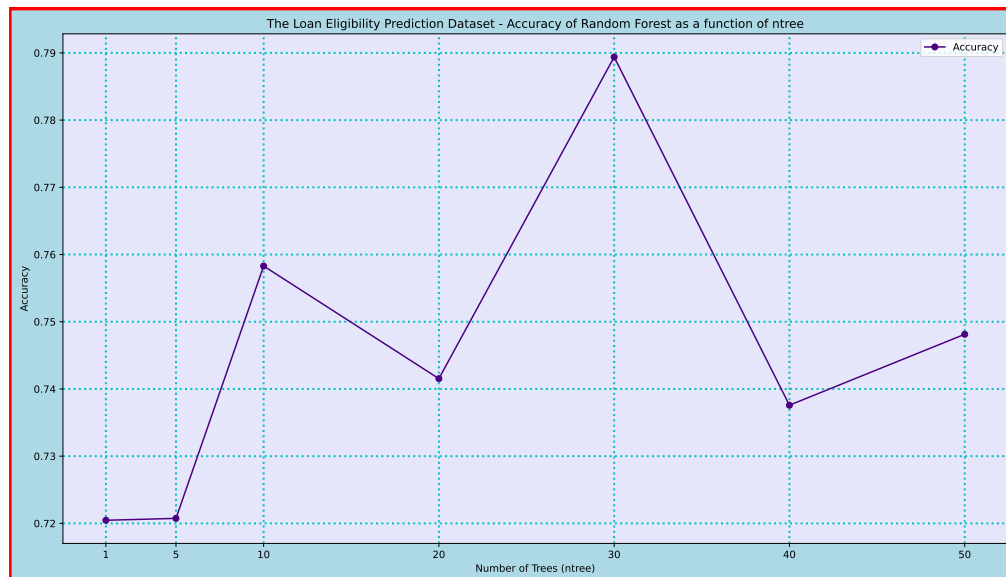


Figure 8: The Loan Eligibility Prediction Dataset - Accuracy of the random forest as a function of $ntree$

Precision of the random forest as a function of $ntree$ for the Loan Eligibility Prediction Dataset

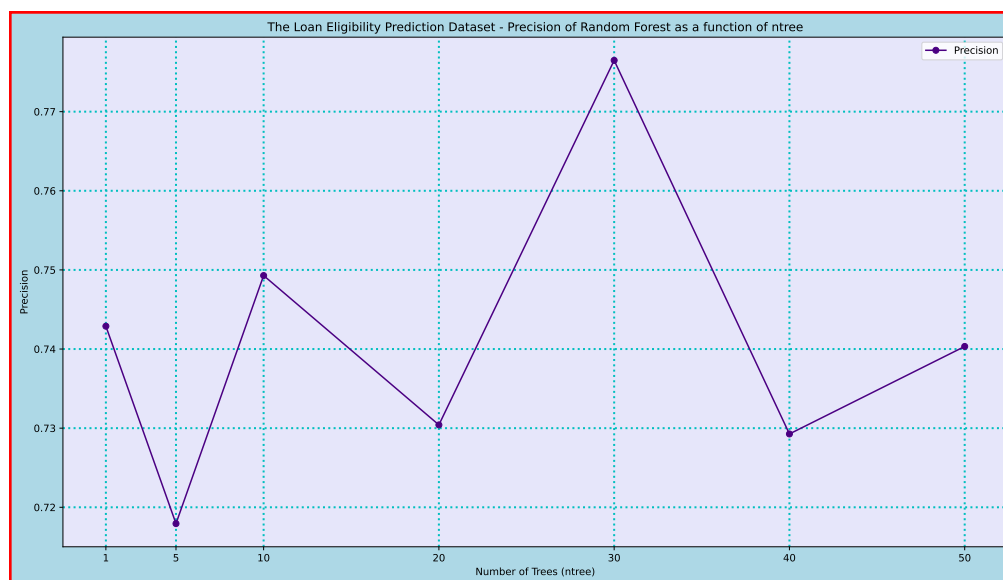


Figure 9: The Loan Eligibility Prediction Dataset - Precision of the random forest as a function of $ntree$

Recall of the random forest as a function of $ntree$ for the Loan Eligibility Prediction Dataset

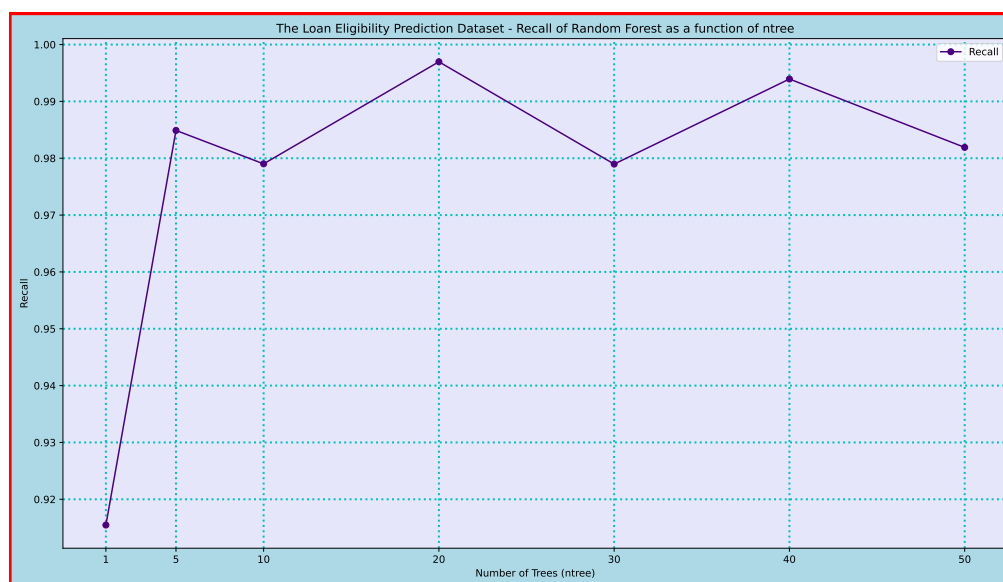


Figure 10: The Loan Eligibility Prediction Dataset - Recall of the random forest as a function of $ntree$

F1 Score of the random forest as a function of $ntree$ for the Loan Eligibility Prediction Dataset

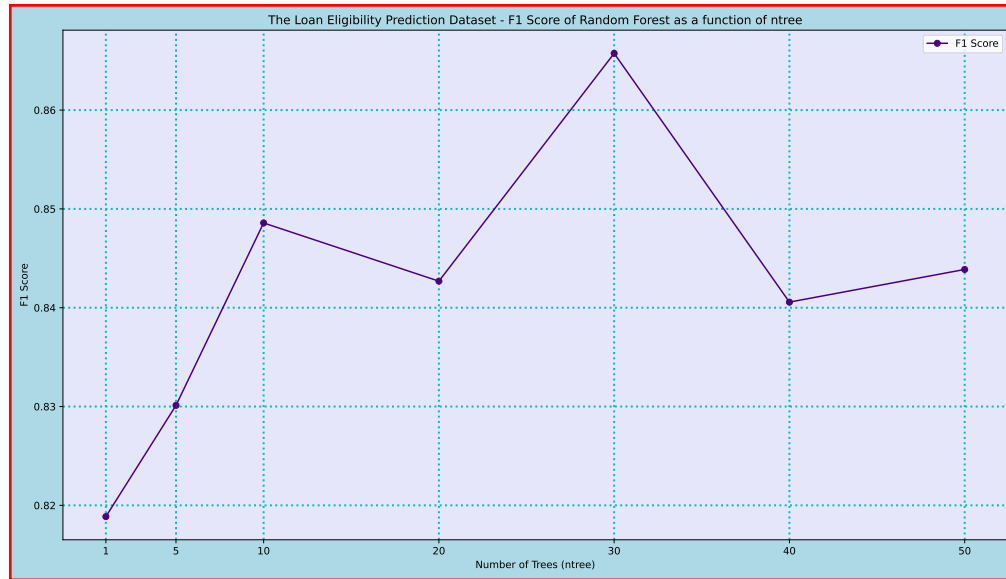


Figure 11: The Loan Eligibility Prediction Dataset - F1 Score of the random forest as a function of $ntree$

- For each metric being evaluated (and for each dataset), discuss which value of $ntree$ you would select if you were to deploy this classifier in real life. Explain your reasoning.

The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

Accuracy - If I were to deploy this classifier in real life, I would choose an $ntree = 10$ for the accuracy metric for the WDBC dataset. For $ntree = 10$, the accuracy value is 95.9569%. Beyond this point, the accuracy remains around 96%. We notice minor improvements in accuracy beyond $ntree = 10$. Additional trees beyond $ntree = 10$ do not provide meaningful improvement but increase the training cost. Beyond $ntree = 10$, the training time actually increases and does not provide any training cost benefits. The model is able to correctly classify 96% of the instances with $ntree = 10$.

Precision - If I were to deploy this classifier in real life, I would choose $ntree = 10$ for the precision metric for the WDBC dataset. For $ntree = 10$, the precision value is 96.4104%. Beyond this point, the precision fluctuates, reaching a maximum value of 96.7256% for $ntree = 40$. Moreover, at $ntree = 30$ and $ntree = 50$, the precision value falls to 95.8709% and 95.9616% respectively. Additional trees beyond $ntree = 10$ do not provide meaningful improvement but increase the training cost. Beyond $ntree = 10$, the training cost and time actually increase and does not provide any training cost benefits. Approximately 96.4104% of the positive predictions are correct for $ntree = 10$.

Recall - If I were to deploy this classifier in real life, I would choose $ntree = 20$ for the recall metric for the WDBC dataset. For $ntree = 20$, the recall value is 98.0399%. The recall value fluctuates beyond $ntree = 20$ and reaches a maximum value of 98.8732% for $ntree = 50$. The recall value is 97.2144% and 97.7582% for $ntree = 30$ and $ntree = 40$, respectively. The increase in recall from $ntree = 20$ to $ntree = 50$ is marginal and only results in increased training and prediction costs. Out of all positive instances, the model is able to correctly predict 98.0399% as positive for $ntree = 20$. This is the chosen optimal point for the recall metric, balancing cost and performance.

F1 Score - If I were to deploy this classifier in real life, I would choose $ntree = 20$ for the F1 Score metric for the WDBC dataset. The F1 Score increases from $ntree = 1$ to $ntree = 20$ and reaches a value of 97.0987%. Beyond this point, the F1 Score fluctuates and reaches a maximum value of 97.3843%

for $ntree = 50$. The difference in F1 Score from $ntree = 20$ to $ntree = 50$ is minimal. Additional trees beyond $ntree = 20$ do not provide meaningful improvement but increase the training cost. Beyond $ntree = 20$, the training time actually increases and does not provide any training cost or performance benefits. Thus, $ntree = 20$ provides a suitable balance between F1 Score and the performance of the model.

The Loan Eligibility Prediction Dataset

Accuracy - If I were to deploy this classifier in real life, I would choose an $ntree$ value of 30 for the accuracy metric for the Loan dataset. For $ntree = 30$, the accuracy value is 78.9391%. The value of $ntree$ is highest at 30. The accuracy increases from $ntree = 1$ to $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the accuracy has fallen to 73.7584% and 74.8134%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost. The $ntree$ value of 30 is the optimal point for accuracy for the Loan dataset. Adding trees beyond $ntree = 30$ leads to overfitting and instability.

Precision - If I were to deploy this classifier in real life, I would choose $ntree = 30$ for the precision metric for the Loan dataset. For $ntree = 30$, the precision value is 77.6486%. The value of $ntree$ is highest at 30. The precision fluctuates from $ntree = 1$ to $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the precision has fallen to 72.9269% and 74.0327%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost. The $ntree$ value of 30 is the optimal point for precision for the Loan dataset. Adding trees beyond $ntree = 30$ leads to overfitting and instability.

Recall - If I were to deploy this classifier in real life, I would choose $ntree = 20$ for the recall metric for the Loan dataset. For $ntree = 20$, the recall value is 99.697%. The recall value is highest at $ntree = 20$. The recall value fluctuates from $ntree = 5$ to $ntree = 50$. Out of all positive instances, the model is able to correctly predict 99.697% as positive for $ntree = 20$. This is the chosen optimal point for the recall metric, balancing cost and performance.

F1 Score - If I were to deploy this classifier in real life, I would choose $ntree = 30$ for the F1 Score metric for the Loan dataset. The F1 Score reaches a maximum value of 86.5744% at $ntree = 30$. The F1 Score increases from $ntree = 1$ to $ntree = 30$ and then drops after $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the F1 Score has fallen to 84.0567% and 84.3873%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost. The $ntree$ value of 30 is the optimal point for F1 Score for the Loan dataset. Adding trees beyond $ntree = 30$ leads to overfitting and instability.

5. Discuss, at a high level, which metrics were more directly affected by changes in the value of $ntree$ and, more generally, how these changes influenced the performance of your algorithm. For example, was the accuracy of the random forest particularly sensitive to increasing $ntree$ beyond a certain value? Was the F1 score a “harder” metric to optimize, possibly requiring a larger number of trees in the ensemble? Is there a point beyond which adding more trees does not improve performance (or even makes it worse)?

The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset

Solution: All four metrics - accuracy, precision, recall and F1 Score increased from $ntree = 1$ to $ntree = 20$, beyond which the curve plateaued with minor fluctuations for WDBC dataset. Recall was more affected with increase in $ntree$. The Recall consistently improved with more trees. The accuracy and precision plateaued after $ntree = 10$. The F1 Score proved to be the harder metrics to optimize requiring more trees than accuracy and precision.

Accuracy: For $ntree = 10$, the accuracy value is 95.9569%. Beyond this point, the accuracy remains around 96%. We notice minor improvements in accuracy beyond $ntree = 10$. Additional trees beyond $ntree = 10$ do not provide meaningful improvement but increase the training cost.

Precision: For $ntree = 10$, the precision value is 96.4104%. Beyond this point, the precision fluctuates, reaching a maximum value of 96.7256% for $ntree = 40$. Moreover, at $ntree = 30$ and $ntree = 50$, the

precision value falls to 95.8709% and 95.9616% respectively. Additional trees beyond $ntree = 10$ do not provide meaningful improvement but increase the training cost.

Recall: For $ntree = 20$, the recall value is 98.0399%. The recall value fluctuates beyond $ntree = 20$ and reaches a maximum value of 98.8732% for $ntree = 50$. The recall value is 97.2144% and 97.7582% for $ntree = 30$ and $ntree = 40$, respectively.

F1 Score: The F1 Score increases from $ntree = 1$ to $ntree = 20$ and reaches a value of 97.0987%. Beyond this point, the F1 Score fluctuates and reaches a maximum value of 97.3843% for $ntree = 50$. The difference in F1 Score from $ntree = 20$ to $ntree = 50$ is minimal. Additional trees beyond $ntree = 20$ do not provide meaningful improvement but increase the training cost.

Adding more trees beyond $ntree = 20$ does not provide any significant performance improvement. Beyond $ntree = 20$ the performance improvements of all the metrics are marginal and performance fluctuates. In some cases, such as precision at $ntree = 30$, performance even decreases with more trees.

The Loan Eligibility Prediction Dataset

Solution: Accuracy, Precision and F1 Score showed drastic fluctuations with an increase in $ntrees$. Accuracy, Precision and F1 Score reached a maximum value at $ntree = 30$ beyond which it started dropping drastically. Only recall plateaued after $ntree = 5$ and reached a maximum at $ntree = 20$.

Accuracy: For $ntree = 30$, the accuracy value is 78.9391%. The value of $ntree$ is highest at 30. The accuracy increases from $ntree = 1$ to $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the accuracy has fallen to 73.7584% and 74.8134%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost.

Precision: For $ntree = 30$, the precision value is 77.6486%. The value of $ntree$ is highest at 30. The precision fluctuates from $ntree = 1$ to $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the precision has fallen to 72.9269% and 74.0327%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost.

Recall: For $ntree = 20$, the recall value is 99.697%. The recall value is highest at $ntree = 20$. The recall value fluctuates from $ntree = 5$ to $ntree = 50$.

F1 Score: The F1 Score reaches a maximum value of 86.5744% at $ntree = 30$. The F1 Score increases from $ntree = 1$ to $ntree = 30$ and then drops after $ntree = 30$. For $ntree = 40$ and $ntree = 50$, the F1 Score has fallen to 84.0567% and 84.3873%, respectively. Increasing the trees beyond $ntree = 30$ results in a decrease in performance, along with additional training and prediction cost.

Adding more trees beyond $ntree = 30$ results in a significant drop in performance of Accuracy, Precision and F1 Score. The performance of the model becomes worse when adding trees beyond $ntree = 30$.

Extra Credit Questions

There are two ways in which you may receive extra credit in this homework.

[QE.1] **Extra points (10 Points)** Analyze a third dataset: the **Raisins Dataset**. The goal here is to use morphological features extracted from raisin images to classify each instance as either a Kecimen or a Besni raisin variety. The dataset contains 900 instances. Each instance is described by 7 *numerical* features, and the dataset contains two classes. You should present the same analyses and graphs discussed above.

This dataset can be found in the “*Supporting Files*” zip file for this homework on Canvas.

The Raisin Dataset

Accuracy, Precision, Recall and F1 Score of the Random Forest for the Raisin Dataset for each value of *ntree*

```
The Raisin Dataset - ntree parameter = 1
{'Accuracy': 0.807778, 'Precision': 0.829283, 'Recall': 0.777778, 'F1 Score': 0.801578}
The Raisin Dataset - ntree parameter = 5
{'Accuracy': 0.856667, 'Precision': 0.875018, 'Recall': 0.833333, 'F1 Score': 0.853075}
The Raisin Dataset - ntree parameter = 10
{'Accuracy': 0.86, 'Precision': 0.897012, 'Recall': 0.813333, 'F1 Score': 0.853105}
The Raisin Dataset - ntree parameter = 20
{'Accuracy': 0.855556, 'Precision': 0.895405, 'Recall': 0.806667, 'F1 Score': 0.847908}
The Raisin Dataset - ntree parameter = 30
{'Accuracy': 0.87, 'Precision': 0.895008, 'Recall': 0.84, 'F1 Score': 0.865977}
The Raisin Dataset - ntree parameter = 40
{'Accuracy': 0.867778, 'Precision': 0.904038, 'Recall': 0.824444, 'F1 Score': 0.861638}
The Raisin Dataset - ntree parameter = 50
{'Accuracy': 0.86, 'Precision': 0.897937, 'Recall': 0.815556, 'F1 Score': 0.852739}
```

Figure 12: Performance metrics - Accuracy, Precision, Recall and F1 Score for Raisin dataset for different values of *ntree*

Accuracy of the random forest as a function of $ntree$ for the Raisin Dataset

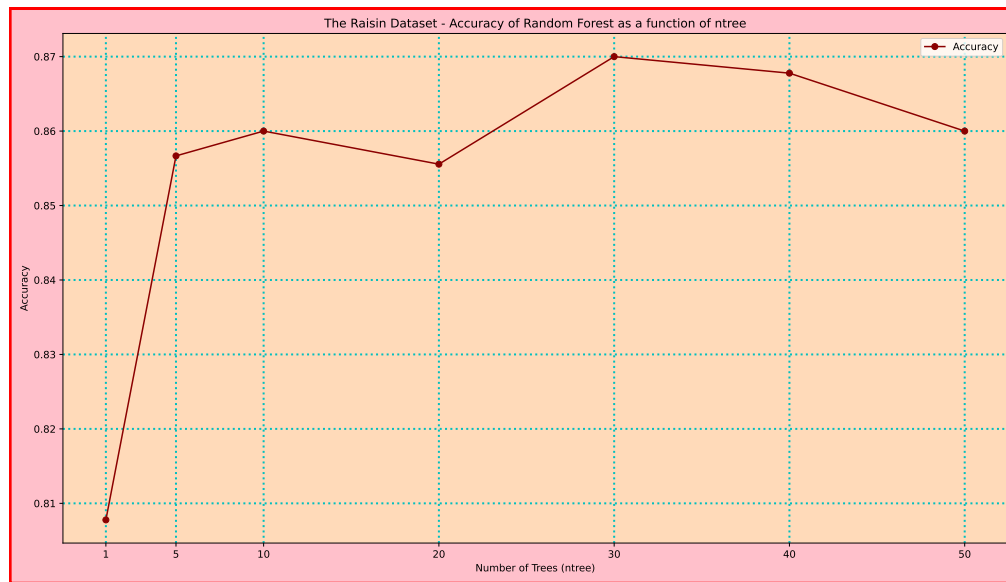


Figure 13: The Raisin Dataset - Accuracy of the random forest as a function of $ntree$

Accuracy: If I were to deploy this classifier in real life, I would choose $ntree = 30$ for the Accuracy metric for the Raisin dataset. The Accuracy reaches a maximum value of 87% at $ntree = 30$. The accuracy increases from $ntree = 1$ to $ntree = 30$ beyond which it started falling slightly. The value of the Accuracy at $ntree = 40$ and $ntree = 50$ are 86.7778% and 86% respectively. Thus $ntree = 30$ is the optimal point for performance and training cost.

Precision of the random forest as a function of $ntree$ for the Raisin Dataset

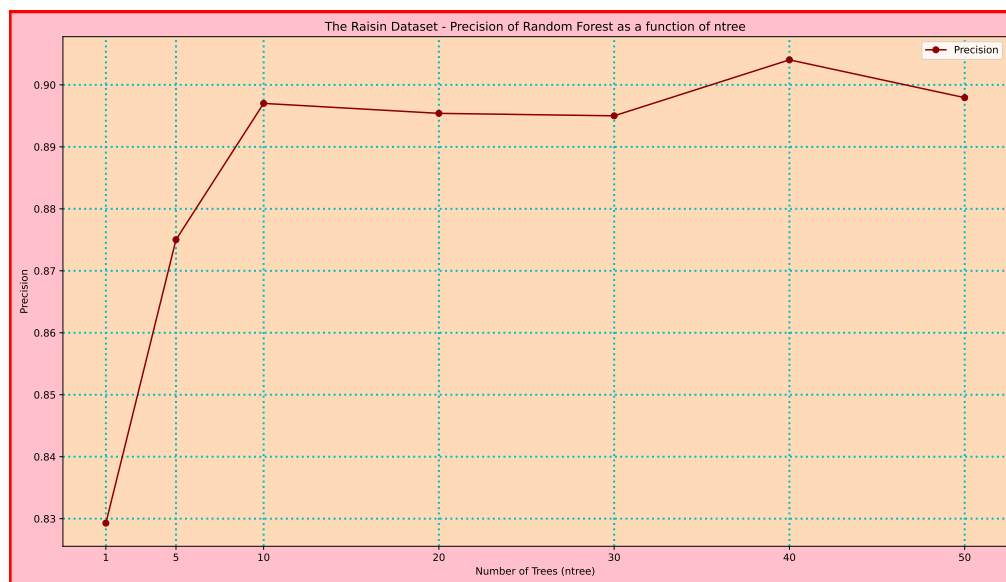


Figure 14: The Raisin Dataset - Precision of the random forest as a function of $ntree$

Precision: If I were to deploy this classifier in real life, I would choose $ntree = 10$ for the Precision metric for the Raisin dataset. The Precision reaches a value of 89.7012% at $ntree = 10$. Beyond which the Precision started plateauing. The Precision reaches a maximum value of 90.4038% at $ntree = 40$. Thus $ntree = 10$ is the optimal point for performance and training cost.

Recall of the random forest as a function of $ntree$ for the Raisin Dataset

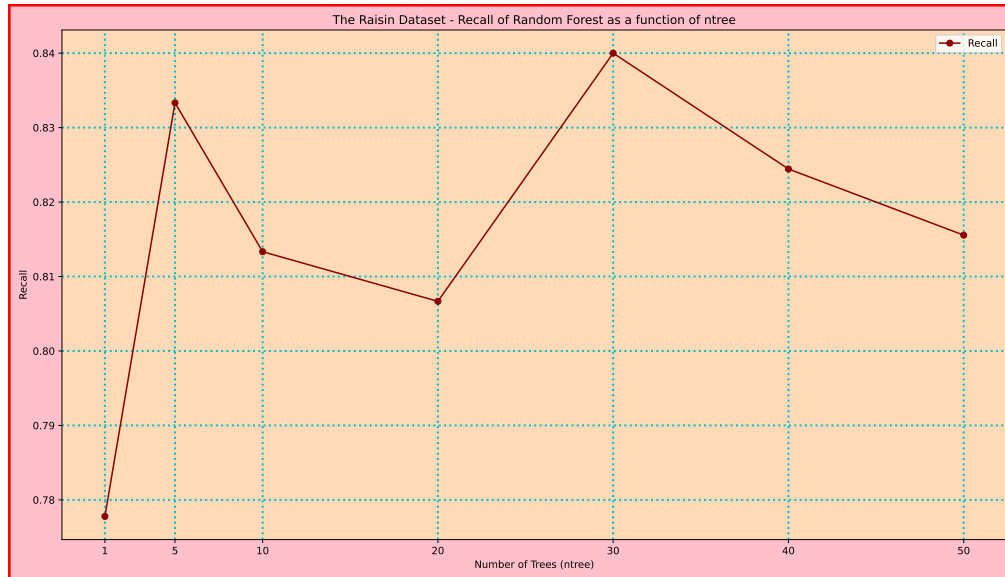


Figure 15: The Raisin Dataset - Recall of the random forest as a function of $ntree$

Recall: If I were to deploy this classifier in real life, I would choose $ntree = 30$ for the Recall metric for the Raisin dataset. The Recall reaches a maximum value of 84% at $ntree = 30$. The Recall drops from $ntree = 5$ to $ntree = 20$ and then reaches a maximum value at $ntree = 30$ and again started falling at $ntree = 40$ and $ntree = 50$. Thus $ntree = 30$ is the optimal point for performance and training cost.

F1 Score of the random forest as a function of n_{tree} for the Raisin Dataset

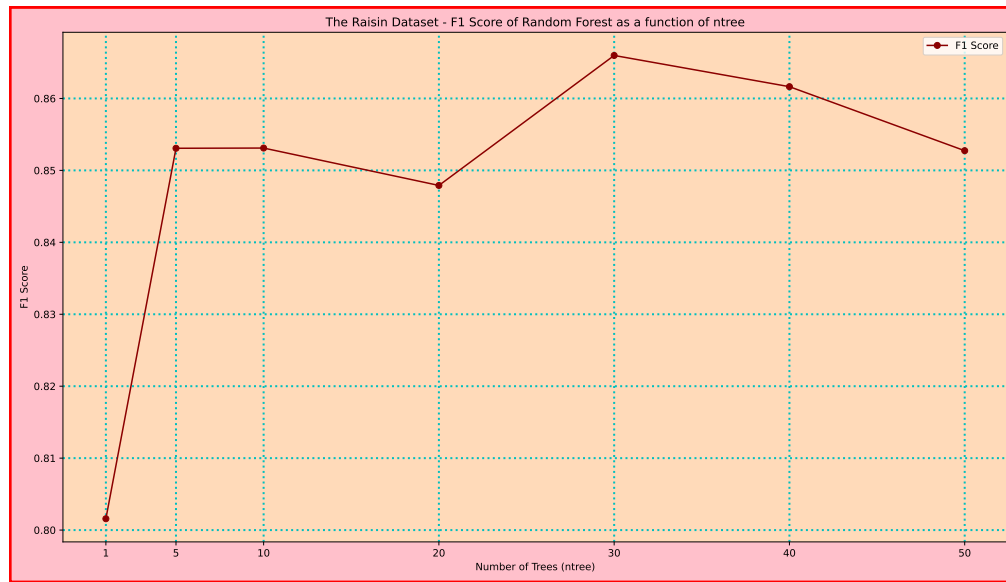


Figure 16: The Raisin Dataset - F1 Score of the random forest as a function of n_{tree}

F1 Score: If I were to deploy this classifier in real life, I would choose $n_{tree} = 30$ for the F1 Score metric for the Raisin dataset. The F1 Score reaches a maximum value of 86.5977% at $n_{tree} = 30$. The F1 Score drops from $n_{tree} = 5$ to $n_{tree} = 20$ and then reaches a maximum value at $n_{tree} = 30$ and again started falling at $n_{tree} = 40$ and $n_{tree} = 50$. Thus $n_{tree} = 30$ is the optimal point for performance and training cost.

Adding more trees beyond $n_{tree} = 30$ does not provide any significant performance improvement. Beyond $n_{tree} = 30$ the performance improvements of all the metrics are marginal and performance fluctuates.

[QE.2] Extra points (10 Points) Analyze a fourth dataset: the **Titanic Dataset**. On April 15, 1912, the largest passenger liner ever made collided with an iceberg during her maiden voyage. When the Titanic sank, it killed 1502 out of 2224 passengers and crew. One of the reasons the shipwreck resulted in such loss of life was that there were not enough lifeboats for the passengers and crew. Although luck was involved in surviving the sinking, some groups of people were more likely to survive than others. The goal here is to predict whether a given person was likely to survive this tragedy. The dataset contains data for 887 real passengers of the Titanic. Each row represents one person. The columns describe different attributes of the person, including whether they survived (the class/label), their age, their passenger class, sex, and the fare they paid. Notice that this dataset combines *both numerical and categorical attributes*. You should present the same analyses and graphs discussed above.

This dataset can be found in the “Supporting Files” zip file for this homework on Canvas.

```

The Titanic Dataset - ntree=1
{'Accuracy': 0.772361, 'Precision': 0.766259, 'Recall': 0.640111, 'F1 Score': 0.686088}
The Titanic Dataset - ntree=5
{'Accuracy': 0.793646, 'Precision': 0.834283, 'Recall': 0.593606, 'F1 Score': 0.685535}
The Titanic Dataset - ntree=10
{'Accuracy': 0.801619, 'Precision': 0.829024, 'Recall': 0.61445, 'F1 Score': 0.703315}
The Titanic Dataset - ntree=20
{'Accuracy': 0.790288, 'Precision': 0.852979, 'Recall': 0.564109, 'F1 Score': 0.670043}
The Titanic Dataset - ntree=30
{'Accuracy': 0.795956, 'Precision': 0.852635, 'Recall': 0.576257, 'F1 Score': 0.684613}
The Titanic Dataset - ntree=40
{'Accuracy': 0.807173, 'Precision': 0.829133, 'Recall': 0.628389, 'F1 Score': 0.713655}
The Titanic Dataset - ntree=50
{'Accuracy': 0.800482, 'Precision': 0.869716, 'Recall': 0.579071, 'F1 Score': 0.686915}

```

Figure 17: Performance metrics - Accuracy, Precision, Recall and F1 Score for Titanic dataset for different values of *ntree*

The Titanic Dataset

Accuracy, Precision, Recall and F1 Score of the Random Forest for the Titanic Dataset for each value of *ntree*

Accuracy of the random forest as a function of *ntree* for the Titanic Dataset

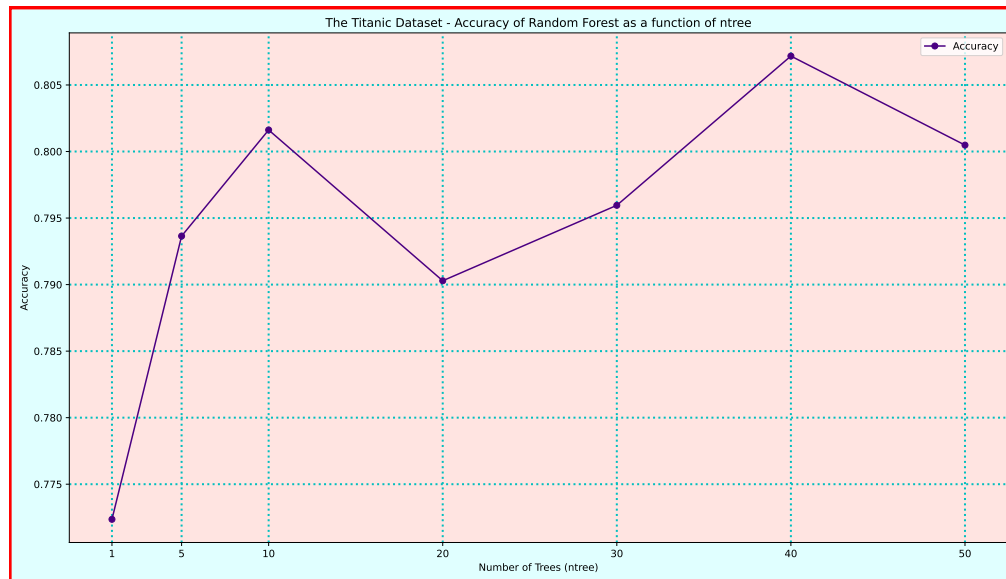


Figure 18: The Titanic Dataset - Accuracy of the random forest as a function of *ntree*

Accuracy: If I were to deploy this classifier in real life, I would choose *ntree* = 40 for the Accuracy metric for the Titanic dataset. The Accuracy reaches a maximum value of 80.7173% at *ntree* = 40. The accuracy increases from *ntree* = 1 to *ntree* = 10 and then it falls at *ntree* = 20. Then it again started rising at *ntree* = 30 and *ntree* = 40. Thus *ntree* = 40 is the optimal point for performance and training cost.

Precision of the random forest as a function of $ntree$ for the Titanic Dataset

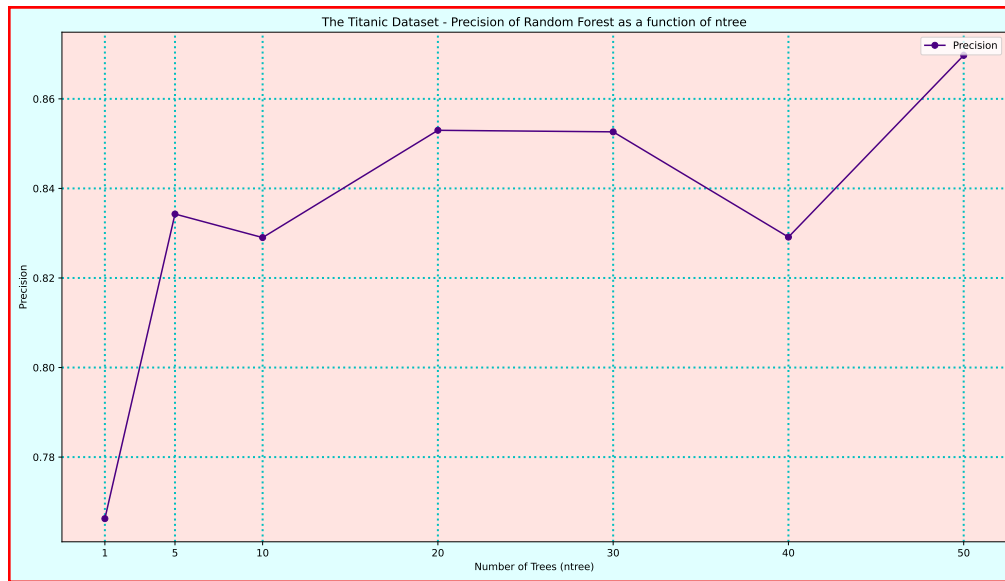


Figure 19: The Titanic Dataset - Precision of the random forest as a function of $ntree$

Precision: If I were to deploy this classifier in real life, I would choose $ntree = 50$ for the Precision metric for the Titanic dataset. The Precision reaches a maximum value of 86.9716% at $ntree = 50$. The precision increases from $ntree = 1$ to $ntree = 5$ and then falls at $ntree = 10$ and then rises at $ntree = 20$ and $ntree = 30$ and then dips at $ntree = 40$ and rises at $ntree = 50$

Recall of the random forest as a function of $ntree$ for the Titanic Dataset

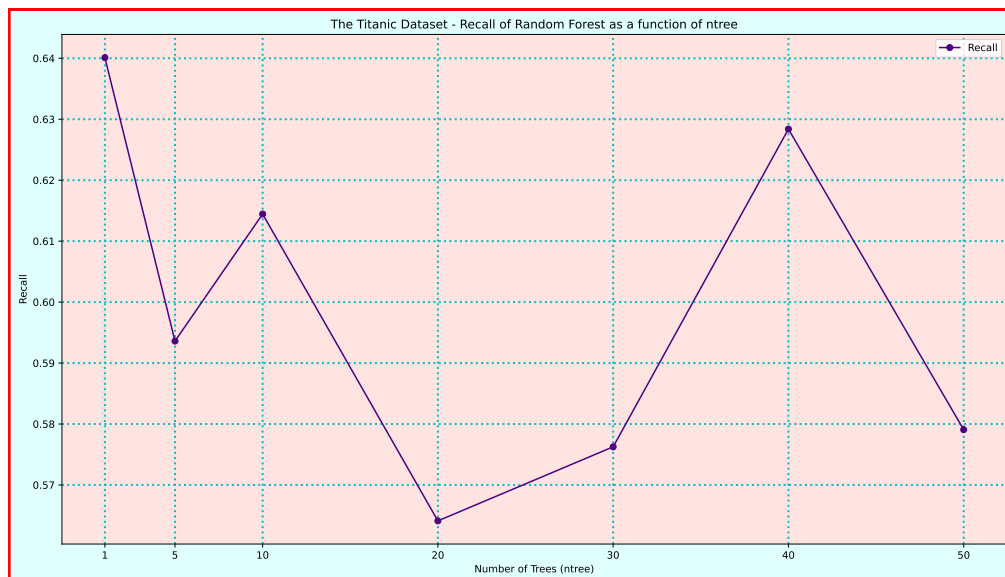


Figure 20: The Titanic Dataset - Recall of the random forest as a function of $ntree$

Recall: If I were to deploy this classifier in real life, I would choose $ntree = 40$ for the Recall metric for the Titanic dataset. The Recall reaches a value of 62.8389% at $ntree = 40$. Thus $ntree = 40$ is the optimal point

for performance and training cost.

F1 Score of the random forest as a function of $ntree$ for the Titanic Dataset

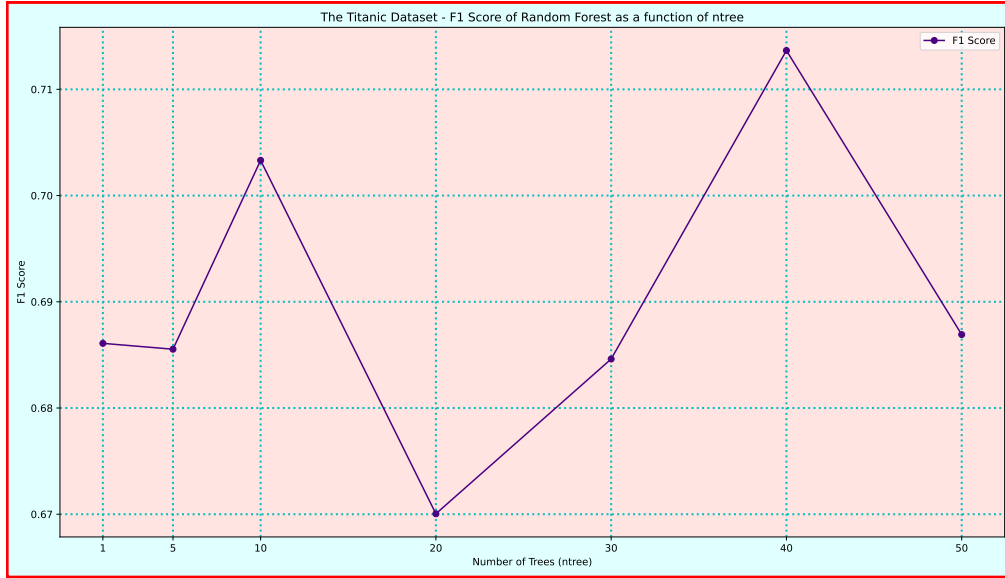


Figure 21: The Titanic Dataset - F1 Score of the random forest as a function of $ntree$

F1 Score: If I were to deploy this classifier in real life, I would choose $ntree = 40$ for the F1 Score metric for the Titanic dataset. The F1 Score reaches a value of 71.3655% at $ntree = 40$. Thus $ntree = 40$ is the optimal point for performance and training cost.

Adding more trees beyond $ntree = 40$ results in a significant drop in performance of Accuracy, Recall and F1 Score. The performance of the model becomes worse when adding trees beyond $ntree = 40$.